

LAPORAN TUGAS 2 (1 - 2)

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Fagil Lola Prihernando (2410651031)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Perkembangan teknologi komputasi modern menuntut adanya pemahaman mendalam tentang konsep manajemen proses dan komunikasi antar proses dalam sistem operasi. Implementasi *multi-processing* dan *Inter-Process Communication* (IPC) menjadi fundamental dalam pengembangan aplikasi yang efisien dan responsif, khususnya pada sistem operasi berbasis UNIX. Praktik manajemen proses memungkinkan eksekusi tugas secara paralel, mengoptimalkan penggunaan sumber daya sistem, sementara mekanisme IPC memfasilitasi pertukaran data antar proses yang berjalan secara independen. Kedua konsep ini sangat krusial dalam pengembangan aplikasi real-time seperti sistem terdistribusi, aplikasi server-client, dan platform kolaboratif. Melalui implementasi langsung menggunakan bahasa pemrograman C, dapat dipelajari secara komprehensif bagaimana sistem operasi mengelola eksekusi proses secara konkuren serta mengoordinasikan komunikasi antar proses dengan menjaga konsistensi data dan menghindari *race condition*. Pemahaman ini menjadi dasar bagi pengembangan aplikasi yang lebih kompleks di masa depan.

1.2 RUMUSAN MASALAH

1. Bagaimana implementasi manajemen proses paralel menggunakan sistem *fork* dan *pipe* untuk menjalankan tugas-tugas komputasi secara simultan?
2. Bagaimana merancang mekanisme IPC melalui *shared memory* dan *semaphore* untuk membangun aplikasi chat room real-time yang mampu menangani multiple users?
3. Bagaimana mengukur dan menganalisis performa eksekusi proses paralel serta efektivitas komunikasi antar proses dalam pertukaran data real-time?

1.3 TUJUAN

- 1 Mengimplementasikan sistem manajemen proses yang dapat mengeksekusi tiga tugas berbeda (sorting, searching, calculation) secara paralel menggunakan *child processes*
- 2 Membangun aplikasi chat room dengan mekanisme IPC yang mampu menyinkronkan komunikasi antar pengguna melalui *shared memory* dan *semaphore*
- 3 Menganalisis efisiensi waktu eksekusi proses paralel dan mengevaluasi kehandalan sistem komunikasi real-time antar proses

BAB II

PEMBAHASAN

2.1 PRAKTIK TUGAS 1

2.1.1 Sistem Manajemen Proses

Dalam implementasi sistem manajemen proses paralel menggunakan bahasa C, program utama berperan sebagai *parent process* yang melakukan *fork* untuk menciptakan tiga *child processes* independen. Setiap *child process* menjalankan tugas spesifik secara konkuren: proses pertama melakukan pengurutan *array* dengan algoritma *Bubble Sort*, proses kedua melakukan pencarian linear terhadap nilai tertentu dalam *array*, sedangkan proses ketiga menghitung statistik dasar seperti nilai total, minimum, maksimum, dan rata-rata. Komunikasi antarproses difasilitasi melalui mekanisme *pipe* untuk mengirimkan hasil eksekusi, termasuk *execution time* dan status keberhasilan, kembali ke *parent process*. Program juga mengintegrasikan fungsi pengukuran waktu menggunakan `gettimeofday()` untuk mencatat durasi eksekusi setiap tugas, yang kemudian ditampilkan dalam laporan terstruktur beserta *Process ID* masing-masing. Hasil eksekusi menunjukkan seluruh proses anak berhasil dijalankan dengan total waktu eksekusi kumulatif hanya 0.000405 detik, mendemonstrasikan efisiensi paralelisasi pada sistem operasi berbasis UNIX.

```

asus@LAPTOP-VRS206DG:~$ touch management_process.c
asus@LAPTOP-VRS206DG:~$ nano management_process.c
asus@LAPTOP-VRS206DG:~$ gcc -o management_process management_process.c
asus@LAPTOP-VRS206DG:~$ ./management_process
=== SISTEM MANAJEMEN PROSES SEDERHANA ===
Parent Process PID: 449

Process 450 (Sorting): Mengurutkan array...
Process 450 (Sorting): Array telah terurut dengan benar
Process 451 (Searching): Mencari nilai dalam array...
Process 451 (Searching): Nilai 42 ditemukan pada index 39

=== PARENT PROCESS MENUNGGU CHILD SELESAI ===
Process 452 (Calculation): Menghitung statistik...
Process 452 (Calculation): Sum=47684, Min=11, Max=996, Average=476.84

=== HASIL EKSEKUSI SEMUA PROSES ===
PID          Task                Time (s)      Status
-----
450          Sorting Array         0.000165      Success
451          Searching Value       0.000109      Success
452          Calculation Stats    0.000131      Success

=== STATISTIK KESELURUHAN ===
Total processes: 3
Total execution time: 0.000405 seconds
Parent Process PID: 449
Semua child processes telah selesai dieksekusi!
asus@LAPTOP-VRS206DG:~$ |

```

2.1.2 Penjelasan Proses Kompilasi dan Eksekusi:

1. Penulisan Kode Program

- File sumber `management_process.c` dibuat menggunakan text editor nano
- Kode program ditulis dalam bahasa C dengan implementasi multi-processing

2. Proses Kompilasi

- Program dikompilasi menggunakan GCC compiler dengan perintah: `gcc -o management_process management_process.c`
- Flag `-o` digunakan untuk menentukan nama output executable sebagai `management_process`
- Proses kompilasi mengubah kode sumber menjadi executable binary yang siap dijalankan

3. Struktur Program yang Dikompilasi

- Program menggunakan header files standar: stdio.h, stdlib.h, unistd.h, sys/types.h, sys/wait.h, sys/time.h, time.h, dan string.h
- Mengimplementasikan sistem multi-processing dengan fork dan pipe
- Terdapat tiga fungsi tugas: sorting, searching, dan calculation

4. Inisialisasi Eksekusi

- Program dijalankan dengan perintah: ./management_process
- Parent process dimulai dengan PID 449 (bervariasi tergantung sistem)
- Array acak berukuran 100 elemen dibuat untuk diproses

5. Pembuatan Child Processes

- Parent process membuat 3 child processes menggunakan fork()
- Setiap pipe dibuat untuk komunikasi inter-process communication
- Tugas didistribusikan: Process 450 (sorting), Process 451 (searching), Process 452 (calculation)

6. Eksekusi Paralel

- Setiap child process menjalankan tugasnya secara bersamaan
- Process 450: Mengurutkan array dengan bubble sort dan memverifikasi hasil
- Process 451: Mencari nilai 42 dalam array menggunakan linear search
- Process 452: Menghitung statistik (sum, min, max, average)

7. Koordinasi dan Pengumpulan Hasil

- Parent process menunggu semua child processes menyelesaikan eksekusi
- Hasil dikumpulkan melalui pipe mechanism
- Waktu eksekusi diukur untuk setiap proses

8. Pelaporan Hasil

- Summary report ditampilkan dalam format tabel terstruktur
- Menampilkan PID, nama tugas, waktu eksekusi, dan status untuk setiap proses
- Statistik keseluruhan dihitung: total processes dan total execution time

9. Terminasi Program

- Semua child processes berhasil menyelesaikan tugas
- Total execution time: 0.000405 detik
- Program mengakhiri eksekusi dengan pesan konfirmasi

10. KODE C:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <sys/time.h>

#include <time.h>

#include <string.h>


#define ARRAY_SIZE 100

#define NUM_PROCESSES 3


// Struktur untuk menyimpan hasil proses

typedef struct {
```

```

pid_t pid;

char task_name[50];

double execution_time;

int status;

char result[100];
} ProcessResult;


// Fungsi untuk menghasilkan array acak
void generate_random_array(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] = rand() % 1000;
    }
}


// Fungsi untuk mengukur waktu eksekusi
double get_current_time() {
    struct timeval tv;

    gettimeofday(&tv, NULL);

    return tv.tv_sec + tv.tv_usec / 1000000.0;
}


// Tugas 1: Sorting (Bubble Sort)
void task_sorting(int arr[], int size) {

    printf("Process %d (Sorting): Mengurutkan array...\n", getpid());

```

```

for (int i = 0; i < size - 1; i++) {
    for (int j = 0; j < size - i - 1; j++) {
        if (arr[j] > arr[j + 1]) {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}

```

// Verifikasi hasil sorting

```

int sorted = 1;
for (int i = 0; i < size - 1; i++) {
    if (arr[i] > arr[i + 1]) {
        sorted = 0;
        break;
    }
}

```

```

printf("Process %d (Sorting): Array %s terurut dengan benar\n",
    getpid(), sorted ? "telah" : "tidak");
}

```


// Tugas 2: Searching (Linear Search)

```
void task_searching(int arr[], int size) {
```

```
    printf("Process %d (Searching): Mencari nilai dalam array...\n", getpid());
```

```
    int target = 42; // Nilai yang dicari
```

```
    int found = 0;
```

```
    int position = -1;
```

```
    for (int i = 0; i < size; i++) {
```

```
        if (arr[i] == target) {
```

```
            found = 1;
```

```
            position = i;
```

```
            break;
```

```
        }
```

```
    }
```

```
    if (found) {
```

```
        printf("Process %d (Searching): Nilai %d ditemukan pada index %d\n",
```

```
            getpid(), target, position);
```

```
    } else {
```

```
        printf("Process %d (Searching): Nilai %d tidak ditemukan\n",
```

```
            getpid(), target);
```

```
    }
```

```
}
```

```
// Tugas 3: Calculation (Menghitung statistik)
```

```
void task_calculation(int arr[], int size) {
```

```
    printf("Process %d (Calculation): Menghitung statistik...\n", getpid());
```

```
    int sum = 0;
```

```
    int min = arr[0];
```

```
    int max = arr[0];
```

```
    for (int i = 0; i < size; i++) {
```

```
        sum += arr[i];
```

```
        if (arr[i] < min) min = arr[i];
```

```
        if (arr[i] > max) max = arr[i];
```

```
    }
```

```
    double average = (double)sum / size;
```

```
    printf("Process %d (Calculation): Sum=%d, Min=%d, Max=%d, Average=%.2f\n",
```

```
        getpid(), sum, min, max, average);
```

```
}
```

```
int main() {
```

```
    printf("=== SISTEM MANAJEMEN PROSES SEDERHANA ===\n");
```

```
    printf("Parent Process PID: %d\n\n", getpid());
```

```
int array[ARRAY_SIZE];

generate_random_array(array, ARRAY_SIZE);
```

```
pid_t pids[NUM_PROCESSES];

int pipefd[NUM_PROCESSES][2];

ProcessResult results[NUM_PROCESSES];
```

```
// Buat pipes untuk komunikasi

for (int i = 0; i < NUM_PROCESSES; i++) {

    if (pipe(pipefd[i]) == -1) {

        perror("Pipe creation failed");

        exit(1);

    }

}
```

```
// Buat multiple child processes

for (int i = 0; i < NUM_PROCESSES; i++) {

    pids[i] = fork();

    if (pids[i] == -1) {

        perror("Fork failed");

        exit(1);

    }

}
```

```
if (pids[i] == 0) {  
    // Child process  
  
    close(pipefd[i][0]); // Tutup read end  
  
    double start_time = get_current_time();  
  
    // Eksekusi tugas berdasarkan index  
    switch (i) {  
        case 0:  
            task_sorting(array, ARRAY_SIZE);  
            strcpy(results[i].task_name, "Sorting Array");  
            break;  
        case 1:  
            task_searching(array, ARRAY_SIZE);  
            strcpy(results[i].task_name, "Searching Value");  
            break;  
        case 2:  
            task_calculation(array, ARRAY_SIZE);  
            strcpy(results[i].task_name, "Calculation Stats");  
            break;  
    }  
  
    double end_time = get_current_time();
```

```

// Isi data hasil

results[i].pid = getpid();

results[i].execution_time = end_time - start_time;

results[i].status = 0; // Success


// Kirim hasil ke parent melalui pipe

write(pipefd[i][1], &results[i], sizeof(ProcessResult));

close(pipefd[i][1]);


exit(0);
}
}


// Parent process - mengkoordinasi dan mengumpulkan hasil

printf("\n=== PARENT PROCESS MENUNGGU CHILD SELESAI ===\n");


for (int i = 0; i < NUM_PROCESSES; i++) {

    close(pipefd[i][1]); // Tutup write end di parent


// Baca hasil dari pipe

ProcessResult result;

read(pipefd[i][0], &result, sizeof(ProcessResult));

```

```

results[i] = result;

close(pipefd[i][0]);


// Tunggu child process selesai

int status;

waitpid(pids[i], &status, 0);


if (WIFEXITED(status)) {

    results[i].status = WEXITSTATUS(status);

}

}


// Tampilkan hasil summary

printf("\n=== HASIL EKSEKUSI SEMUA PROSES ===\n");

printf("%-10s %-20s %-12s %-8s\n",

        "PID", "Task", "Time (s)", "Status");

printf("-----\n");


double total_time = 0;

for (int i = 0; i < NUM_PROCESSES; i++) {

    printf("%-10d %-20s %-12.6f %-8s\n",

        results[i].pid,

        results[i].task_name,

        results[i].execution_time,

```

```
        results[i].status == 0 ? "Success" : "Failed");

    total_time += results[i].execution_time;
}

printf("\n=== STATISTIK KESELURUHAN ===\n");
printf("Total processes: %d\n", NUM_PROCESSES);
printf("Total execution time: %.6f seconds\n", total_time);
printf("Parent Process PID: %d\n", getpid());
printf("Semua child processes telah selesai dieksekusi!\n");

return 0;
}
```

2.1 PRAKTIK TUGAS 2

2.2.1 Chat Application dengan IPC

Implementasi sistem *chat room* real-time ini menggunakan mekanisme *Inter-Process Communication* (IPC) yang canggih melalui *shared memory* dan *semaphore* untuk mengoordinasikan komunikasi antar pengguna. Program dikompilasi dengan perintah `gcc chat.c -o chat` yang menghasilkan executable binary, kemudian dijalankan dengan menyertakan parameter username seperti `./chat User1` dan `./chat User2`. Sistem ini memanfaatkan *shared memory segment* dengan kunci unik yang dihasilkan oleh `ftok()` untuk menyimpan struktur data `chat_room` yang berisi array pesan, *message count*, dan *read count*. Mekanisme sinkronisasi dijaga oleh *semaphore binary* yang mengimplementasikan *mutex lock* melalui operasi `semop()` dengan nilai -1 untuk *lock* dan +1 untuk *unlock*, memastikan konsistensi data saat multiple processes mengakses *shared memory* secara bersamaan.

Fungsi `send_message()` mengintegrasikan *timestamp* menggunakan fungsi `localtime()` dan memformat pesan dengan struktur [HH:MM:SS] User: message, sementara algoritma *circular buffer* diterapkan untuk menangani batasan `MAX_MESSAGES` dengan menggeser pesan tertua ketika buffer penuh. Fitur history menampilkan seluruh *chat log* yang tersimpan melalui fungsi `show_history()` dengan tetap mempertahankan mekanisme *synchronization* untuk menghindari *race condition*. Eksekusi program menunjukkan interaksi dua user yang dapat berkomunikasi secara real-time, dimana pesan yang dikirim oleh satu user langsung terlihat pada history user lainnya, membuktikan efektivitas implementasi *shared memory* dalam pertukaran data antar proses. Program diakhiri dengan perintah `exit` yang melakukan *cleanup* resources melalui `shmdt()` untuk melepaskan *shared memory attachment*.

- **Buat file chat.c:**

```
asus@LAPTOP-VRS206DG:~$ nano chat.c
asus@LAPTOP-VRS206DG:~$ touch chatfile
asus@LAPTOP-VRS206DG:~$ gcc chat.c -o chat
asus@LAPTOP-VRS206DG:~$ |
```


- **Terminal 1:**

```
asus@LAPTOP-VRS206DG:~$ ./chat User1
=== CHAT ROOM ===
Welcome User1! (Type 'exit' to quit, 'history' to view messages)
User1> hello fagil
User1> bagaimana kabarmu?
User1> history

=== CHAT HISTORY ===
[21:55:32] User1: hello fagil
[21:55:45] User2: hello fagil2
[21:56:35] User1: bagaimana kabarmu?
[21:56:48] User2: baik
[21:58:40] User2: hello fagil2
[21:58:47] User2: baik
=====
User1> exit
Goodbye User1!
asus@LAPTOP-VRS206DG:~$ |
```

- **Terminal 2:**

```
asus@LAPTOP-VRS206DG:~$ ./chat User2
=== CHAT ROOM ===
Welcome User2! (Type 'exit' to quit, 'history' to view messages)
User2> hello fagil2
User2> baik
User2> exit
Goodbye User2!
asus@LAPTOP-VRS206DG:~$ |
```

2.1.2 Penjelasan Proses Kompilasi dan Eksekusi:

1. Penulisan Kode Program

- File chat.c dibuat menggunakan text editor nano
- Program ditulis dalam bahasa C dengan implementasi IPC (Inter-Process Communication)

2. Proses Kompilasi

- Program dikompilasi dengan perintah: gcc chat.c -o chat
- Flag -o menentukan nama output executable sebagai chat
- Kode dikompilasi menjadi binary executable yang siap dijalankan

3. Inisialisasi Eksekusi

- Program dijalankan dengan parameter username: ./chat User1 atau ./chat User2
- Sistem memvalidasi parameter command-line arguments
- Jika tidak ada username, program menampilkan usage instructions

4. Setup Shared Memory

- Key unik dibuat menggunakan ftok("chatfile", 65)
- Shared memory segment dibuat/diakses dengan shmget()
- Struktur chat_room di-attach ke process address space dengan shmat()
- Inisialisasi pertama: message_count = 0 dan read_count = 0

5. Inisialisasi Semaphore

- Semaphore dibuat dengan semget() untuk sinkronisasi
- Nilai semaphore di-set ke 1 (binary semaphore) menggunakan semctl()
- Berfungsi sebagai mutex lock untuk mengontrol akses ke shared memory

6. Interface Pengguna

- Menampilkan welcome message dan instruksi penggunaan
- Prompt input menunjukkan username: User1> atau User2>
- Mendukung commands: exit untuk keluar, history untuk melihat pesan

7. Mekanisme Pengiriman Pesan

- Input user diproses dengan fgets()
- Newline character di-remove menggunakan strcspn()
- Semaphore di-lock sebelum mengakses shared memory
- Timestamp dibuat dengan time() dan localtime()
- Pesan diformat: [HH:MM:SS] User: message
- Circular buffer implemented untuk menangani batas MAX_MESSAGES

8. Fitur History Chat

- Command history memanggil fungsi show_history()

- Semaphore di-lock selama membaca shared memory
- Menampilkan semua pesan yang tersimpan dalam format terstruktur
- Jika tidak ada pesan, menampilkan "No messages yet"

9. Sinkronisasi Real-time

- Semaphore memastikan hanya satu process yang mengakses shared memory
- Operasi P (wait): sem_op = -1 untuk lock
- Operasi V (signal): sem_op = +1 untuk unlock
- Mencegah race condition pada akses concurrent

10. Terminasi Program

- Command exit mengakhiri loop utama
- Shared memory di-detach dengan shmdt()
- Menampilkan goodbye message
- Shared memory dan semaphore tetap aktif untuk proses lainnya

11. Komunikasi Multi-User

- Multiple instances dapat berjalan bersamaan
- Pesan yang dikirim satu user langsung terlihat di history user lain
- Demonstrasi efektivitas IPC melalui shared memory
- Waktu eksekusi menunjukkan komunikasi real-time antara User1 dan User2

12. KODE C:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
#include <sys/sem.h>
```

```
#include <sys/types.h>
```

```
#include <time.h>
```

```
#define SHM_SIZE 1024
```

```
#define MAX_MESSAGES 10
```

```
#define MESSAGE_SIZE 100
```

```
// Struktur untuk shared memory
```

```
typedef struct {
```

```
    char messages[MAX_MESSAGES][MESSAGE_SIZE];
```

```
    int message_count;
```

```
    int read_count;
```

```
} chat_room;
```

```
// Union untuk semctl
```

```
union semun {
```

```
    int val;
```

```
    struct semid_ds *buf;
```

```
    unsigned short *array;
```

```
};
```

```

// Fungsi untuk mengirim pesan

void send_message(chat_room *room, int sem_id, const char *user, const char *message) {

    struct sembuf sem_op;

    // Lock semaphore

    sem_op.sem_num = 0;

    sem_op.sem_op = -1;

    sem_op.sem_flg = 0;

    semop(sem_id, &sem_op, 1);

    // Format pesan dengan timestamp

    time_t now = time(NULL);

    struct tm *t = localtime(&now);

    char timestamp[20];

    snprintf(timestamp, sizeof(timestamp), "%02d:%02d:%02d", t->tm_hour, t->tm_min, t->tm_sec);

    // Tambahkan pesan ke chat room

    if (room->message_count < MAX_MESSAGES) {

        snprintf(room->messages[room->message_count], MESSAGE_SIZE,

            "[%s] %s: %s", timestamp, user, message);

        room->message_count++;

    } else {

        // Geser pesan lama jika sudah penuh

        for (int i = 0; i < MAX_MESSAGES - 1; i++) {

```

```

        strcpy(room->messages[i], room->messages[i + 1]);
    }

    snprintf(room->messages[MAX_MESSAGES - 1], MESSAGE_SIZE,
        "[%s] %s: %s", timestamp, user, message);
}

// Unlock semaphore

sem_op.sem_num = 0;

sem_op.sem_op = 1;

sem_op.sem_flg = 0;

semop(sem_id, &sem_op, 1);
}

// Fungsi untuk menampilkan history chat

void show_history(chat_room *room, int sem_id) {

    struct sembuf sem_op;

    // Lock semaphore

    sem_op.sem_num = 0;

    sem_op.sem_op = -1;

    sem_op.sem_flg = 0;

    semop(sem_id, &sem_op, 1);

    printf("\n=== CHAT HISTORY ===\n");

```

```

if (room->message_count == 0) {

    printf("No messages yet.\n");

} else {

    for (int i = 0; i < room->message_count; i++) {

        printf("%s\n", room->messages[i]);

    }

}

printf("=====\n");


// Unlock semaphore

sem_op.sem_num = 0;

sem_op.sem_op = 1;

sem_op.sem_flg = 0;

semop(sem_id, &sem_op, 1);

}


int main(int argc, char *argv[]) {

    if (argc != 2) {

        printf("Usage: %s <username>\n", argv[0]);

        printf("Example: %s User1\n", argv[0]);

        return 1;

    }

    char *username = argv[1];

```

```

key_t key = ftok("chatfile", 65);

int shm_id, sem_id;

chat_room *room;

union semun sem_arg;


// Buat shared memory

shm_id = shmget(key, sizeof(chat_room), 0666 | IPC_CREAT);

if (shm_id == -1) {

    perror("shmget failed");

    return 1;

}


// Attach shared memory

room = (chat_room *)shmat(shm_id, NULL, 0);

if (room == (void *)-1) {

    perror("shmat failed");

    return 1;

}


// Inisialisasi shared memory pertama kali

if (room->message_count < 0 || room->message_count > MAX_MESSAGES) {

    room->message_count = 0;

    room->read_count = 0;

}

```



```

// Buat semaphore

sem_id = semget(key, 1, 0666 | IPC_CREAT);

if (sem_id == -1) {

    perror("semget failed");

    return 1;

}


// Inisialisasi semaphore

sem_arg.val = 1;

if (semctl(sem_id, 0, SETVAL, sem_arg) == -1) {

    perror("semctl failed");

    return 1;

}


printf("=== CHAT ROOM ===\n");

printf("Welcome %s! (Type 'exit' to quit, 'history' to view messages)\n", username);


char message[MESSAGE_SIZE];

while (1) {

    printf("%s> ", username);

    fflush(stdout);

    if (fgets(message, sizeof(message), stdin) == NULL) {

```

```
        break;
    }

    // Hapus newline
    message[strcspn(message, "\n")] = 0;

    // Periksa perintah
    if (strcmp(message, "exit") == 0) {
        break;
    } else if (strcmp(message, "history") == 0) {
        show_history(room, sem_id);
    } else if (strlen(message) > 0) {
        send_message(room, sem_id, username, message);
    }
}

// Detach shared memory
shmdt(room);

printf("Goodbye %s!\n", username);

return 0;
}
```

BAB III

PENUTUP

3.1 KESIMPULAN

Berdasarkan implementasi dan analisis yang telah dilakukan, dapat disimpulkan bahwa sistem operasi modern memiliki kemampuan yang powerful dalam menangani manajemen proses dan komunikasi antar proses. Praktik manajemen proses berhasil mendemonstrasikan eksekusi paralel yang efisien dengan waktu eksekusi total hanya 0.000405 detik untuk tiga tugas berbeda, membuktikan optimalisasi sumber daya sistem melalui mekanisme *fork* dan *pipe*. Sementara itu, implementasi IPC pada aplikasi chat room berhasil membuktikan efektivitas *shared memory* dan *semaphore* dalam mengkoordinasikan komunikasi real-time antar pengguna, dengan kemampuan sinkronisasi yang mencegah *race condition* dan menjaga konsistensi data. Kedua praktik ini mengonfirmasi bahwa bahasa C dengan sistem call UNIX memberikan fleksibilitas tinggi dalam pengembangan aplikasi konkuren dan terdistribusi. Hasil eksperimen juga menunjukkan bahwa mekanisme IPC tidak hanya feasible untuk aplikasi sederhana tetapi dapat diskalakan untuk sistem yang lebih kompleks. Pemahaman mendalam tentang konsep ini menjadi fondasi kritis dalam pengembangan sistem operasi, aplikasi distributed computing, dan platform kolaboratif modern yang memerlukan efisiensi tinggi dan real-time responsiveness.